L38: LLMs for Program Analysis: Fuzzing

Software Analysis

IIT Guwahati · Nov 4, 2026

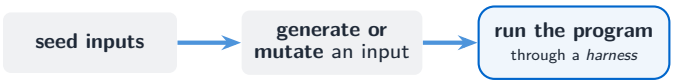
Fuzzing (1/5)

Fuzzing: run the program on many generated inputs, and watch for failures

seed inputs

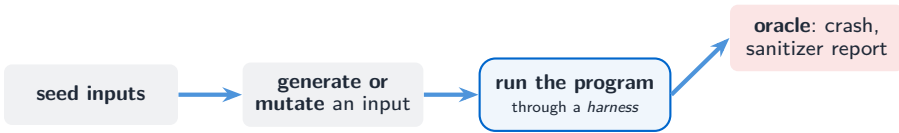
Fuzzing (2/5)

Fuzzing: run the program on many generated inputs, and watch for failures



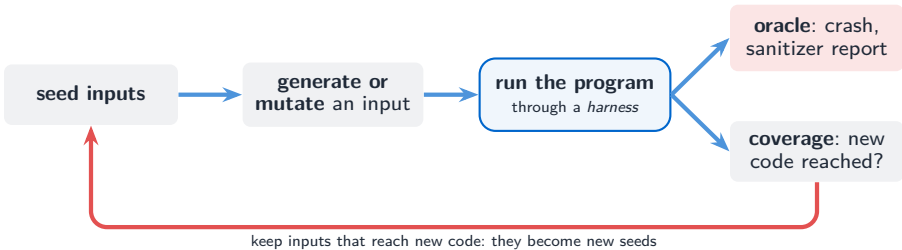
Fuzzing (3/5)

Fuzzing: run the program on many generated inputs, and watch for failures



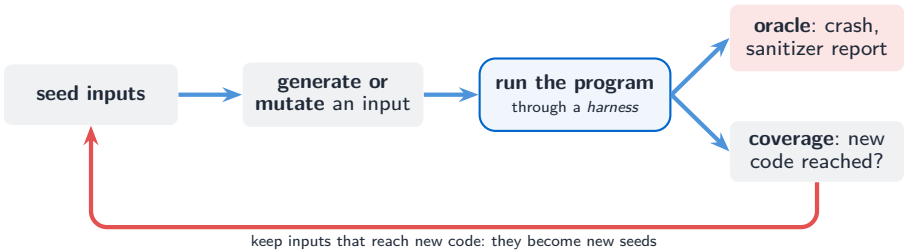
Fuzzing (4/5)

Fuzzing: run the program on many generated inputs, and watch for failures



Fuzzing (5/5)

Fuzzing: run the program on many generated inputs, and watch for failures



Hard parts: **valid structured inputs** (programs, protocols, files) and **harnesses**

A Harness and the Bug It Finds (1/3)

```
5  // Record: 'R' 'E' 'C' <len> <len bytes of payload>
6  int parse_record(const uint8_t* data, size_t size,
7                   char* out, size_t cap) {
8    if (size < 4) return -1;
9    if (data[0] != 'R' || data[1] != 'E' || data[2] != 'C')
10     return -1;
11   size_t len = data[3];
12   if (len > cap) return -1;
13   std::memcpy(out, data + 4, len); // BUG: len > size - 4?
14   return (int)len;
15 }
16
17 extern "C" int LLVMFuzzerTestOneInput(const uint8_t* data,
18                                       size_t size) {
19   char buf[64];
20   parse_record(data, size, buf, sizeof buf);
21   return 0;
22 }
```

A Harness and the Bug It Finds (2/3)

```
5  // Record: 'R' 'E' 'C' <len> <len bytes of payload>
6  int parse_record(const uint8_t* data, size_t size,
7                   char* out, size_t cap) {
8    if (size < 4) return -1;
9    if (data[0] != 'R' || data[1] != 'E' || data[2] != 'C')
10     return -1;
11   size_t len = data[3];
12   if (len > cap) return -1;
13   std::memcpy(out, data + 4, len); // BUG: len > size - 4?
14   return (int)len;
15 }
16
17 extern "C" int LLVMFuzzerTestOneInput(const uint8_t* data,
18                                       size_t size) {
19   char buf[64];
20   parse_record(data, size, buf, sizeof buf);
21   return 0;
22 }
```

Input found by fuzzing: `REC` + byte 0x20 (4 bytes, but `len` = 32)

```
$ ./replay crash.bin
ERROR: AddressSanitizer: heap-buffer-overflow
READ of size 32
  #1 in parse_record(...) parse.cpp:13
  #2 in LLVMFuzzerTestOneInput parse.cpp:20
... is located 0 bytes after 4-byte region
```

Replayed natively with `-fsanitize=address`; addresses removed.

A Harness and the Bug It Finds (3/3)

```
// Record: 'R' 'E' 'C' <len> <len bytes of payload>
int parse_record(const uint8_t* data, size_t size,
                 char* out, size_t cap) {
  if (size < 4) return -1;
  if (data[0] != 'R' || data[1] != 'E' || data[2] != 'C')
    return -1;
  size_t len = data[3];
  if (len > cap) return -1;
  std::memcpy(out, data + 4, len); // BUG: len > size - 4?
  return (int)len;
}

extern "C" int LLVMFuzzerTestOneInput(const uint8_t* data,
                                      size_t size) {
  char buf[64];
  parse_record(data, size, buf, sizeof buf);
  return 0;
}
```

Input found by fuzzing: `REC` + byte 0x20 (4 bytes, but `len` = 32)

```
$ ./replay crash.bin
ERROR: AddressSanitizer: heap-buffer-overflow
READ of size 32
  #1 in parse_record(...) parse.cpp:13
  #2 in LLVMFuzzerTestOneInput parse.cpp:20
... is located 0 bytes after 4-byte region
```

Replayed natively with `-fsanitize=address`; addresses removed.

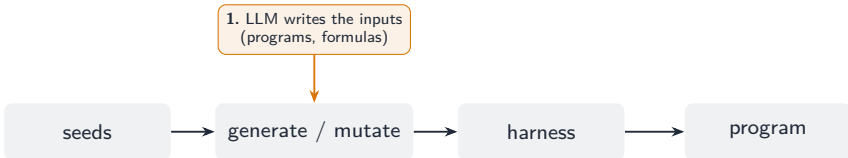
The harness is **written by hand**, per API.
Can an LLM write it? Can it write the **inputs**?

Where Can an LLM Help a Fuzzer? (1/6)



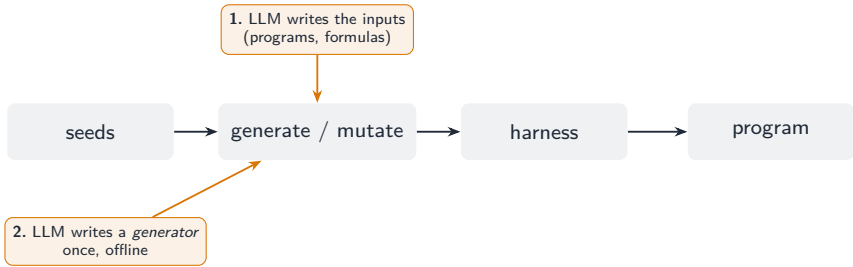
Pradel, Program Analysis, Stuttgart: LLM-based Test Generation

Where Can an LLM Help a Fuzzer? (2/6)

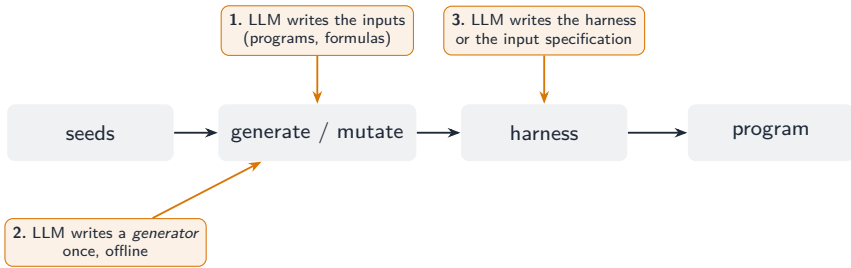


Pradel, Program Analysis, Stuttgart: LLM-based Test Generation

Where Can an LLM Help a Fuzzer? (3/6)

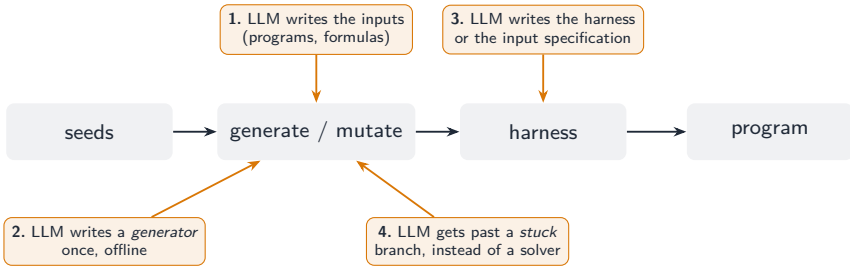


Where Can an LLM Help a Fuzzer? (4/6)

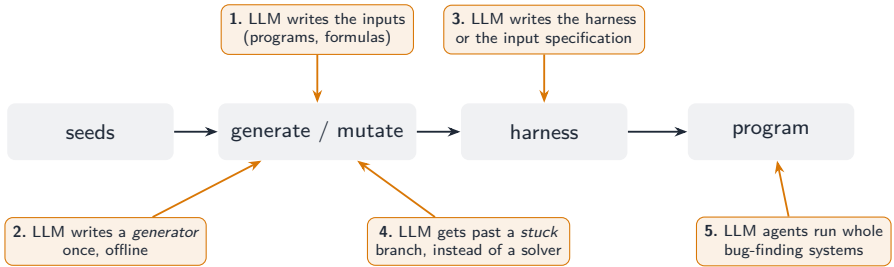


Pradel, Program Analysis, Stuttgart: LLM-based Test Generation

Where Can an LLM Help a Fuzzer? (5/6)



Where Can an LLM Help a Fuzzer? (6/6)



Pradel, Program Analysis, Stuttgart: LLM-based Test Generation

Fuzz4All: Universal Fuzzing (1/5)

Problem: every input language needs its own fuzzer. Can one fuzzer target them all?

user input
docs, examples, specs

Xia, Paltenghi, Tian, Pradel, Zhang, ICSE 2024; Pradel, Program Analysis, Stuttgart: LLM-based Test Generation

Fuzz4All: Universal Fuzzing (2/5)

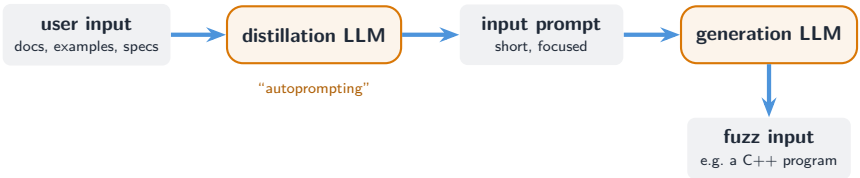
Problem: every input language needs its own fuzzer. Can one fuzzer target them all?



Xia, Paltenghi, Tian, Pradel, Zhang, ICSE 2024; Pradel, Program Analysis, Stuttgart: LLM-based Test Generation

Fuzz4All: Universal Fuzzing (3/5)

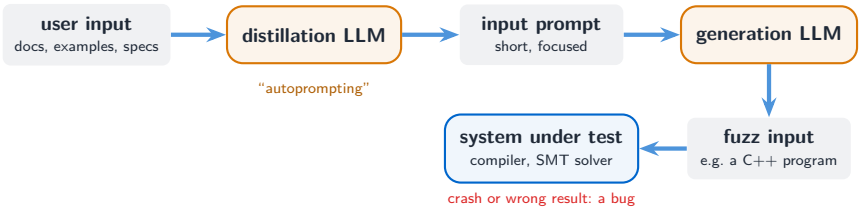
Problem: every input language needs its own fuzzer. Can one fuzzer target them all?



Xia, Paltenghi, Tian, Pradel, Zhang, ICSE 2024; Pradel, Program Analysis, Stuttgart: LLM-based Test Generation

Fuzz4All: Universal Fuzzing (4/5)

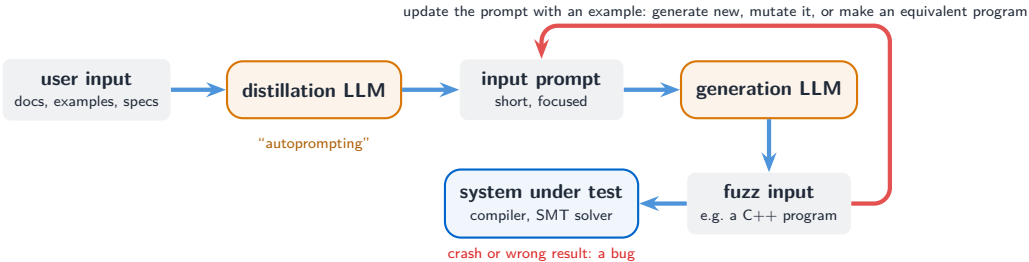
Problem: every input language needs its own fuzzer. Can one fuzzer target them all?



Xia, Paltenghi, Tian, Pradel, Zhang, ICSE 2024; Pradel, Program Analysis, Stuttgart: LLM-based Test Generation

Fuzz4All: Universal Fuzzing (5/5)

Problem: every input language needs its own fuzzer. Can one fuzzer target them all?



Xia, Paltenghi, Tian, Pradel, Zhang, ICSE 2024; Pradel, Program Analysis, Stuttgart: LLM-based Test Generation

Fuzz4All: Results (1/3)

- 9 systems, 6 input languages:
  C, C++, Go, SMT2, Java, Python

Xia, Paltenghi, Tian, Pradel, Zhang, ICSE 2024; Pradel, Program Analysis, Stuttgart: LLM-based Test Generation

Fuzz4All: Results (2/3)

- 9 systems, 6 input languages:
  C, C++, Go, SMT2, Java, Python
- Coverage: **+36.8%** on average over the language-specific fuzzers

Xia, Paltenghi, Tian, Pradel, Zhang, ICSE 2024; Pradel, Program Analysis, Stuttgart: LLM-based Test Generation

Fuzz4All: Results (3/3)

- 9 systems, 6 input languages: C, C++, Go, SMT2, Java, Python
- Coverage: **+36.8%** on average over the language-specific fuzzers
- **98 bugs**, 64 confirmed by the developers as previously unknown

GCC	Clang
Z3	CVC5
OpenJDK	Qiskit

among the systems where bugs were found

Xia, Paltenghi, Tian, Pradel, Zhang, ICSE 2024; Pradel, Program Analysis, Stuttgart: LLM-based Test Generation

Quiz: Fuzzing with LLMs (1/4)

- Why do random bytes rarely find bugs in a C compiler?

- Fuzz4All's oracle: does it need the expected output of each program?

- What is the main cost of calling an LLM for every input?

Quiz: Fuzzing with LLMs (2/4)

- Why do random bytes rarely find bugs in a C compiler?
  They are almost never a valid program, so they stop at the parser.
- Fuzz4All's oracle: does it need the expected output of each program?
- What is the main cost of calling an LLM for every input?

Xia, Paltenghi, Tian, Pradel, Zhang, ICSE 2024; Pradel, Program Analysis, Stuttgart: LLM-based Test Generation

Quiz: Fuzzing with LLMs (3/4)

- Why do random bytes rarely find bugs in a C compiler?
  They are almost never a valid program, so they stop at the parser.
- Fuzz4All's oracle: does it need the expected output of each program?
  No. A crash or a wrong result is enough: a generic oracle.
- What is the main cost of calling an LLM for every input?

Xia, Paltenghi, Tian, Pradel, Zhang, ICSE 2024; Pradel, Program Analysis, Stuttgart: LLM-based Test Generation

Quiz: Fuzzing with LLMs (4/4)

- Why do random bytes rarely find bugs in a C compiler?
  They are almost never a valid program, so they stop at the parser.
- Fuzz4All's oracle: does it need the expected output of each program?
  No. A crash or a wrong result is enough: a generic oracle.
- What is the main cost of calling an LLM for every input?
  Speed: an LLM call takes far longer than one run of the program.

Xia, Paltenghi, Tian, Pradel, Zhang, ICSE 2024; Pradel, Program Analysis, Stuttgart: LLM-based Test Generation

The LLM Writes the Generator (1/5)

Call the LLM **once**, offline; then fuzz at full speed with what it wrote

Tool		Target	What the LLM writes	Result
LegoFuzz 2025	OOPSLA	GCC, LLVM	small, UB-free C functions	66 bugs, almost half miscompilations

Ni and Li, OOPSLA 2025; Sun, Yang, Zhou, ASPLOS 2026; Zhang et al., USENIX Security 2025; Chen, Dolan-Gavitt, Lin, USENIX Security 2025

The LLM Writes the Generator (2/5)

Call the LLM **once**, offline; then fuzz at full speed with what it wrote

Tool	Target	What the LLM writes	Result
LegoFuzz OOPSLA 2025	GCC, LLVM	small, UB-free C functions	66 bugs, almost half miscompilations
Once4All ASPLOS 2026	Z3, cvc5	grammars and term generators	43 confirmed bugs, 40 fixed

Ni and Li, OOPSLA 2025; Sun, Yang, Zhou, ASPLOS 2026; Zhang et al., USENIX Security 2025; Chen, Dolan-Gavitt, Lin, USENIX Security 2025

The LLM Writes the Generator (3/5)

Call the LLM **once**, offline; then fuzz at full speed with what it wrote

Tool	Target	What the LLM writes	Result
LegoFuzz OOPSLA 2025	GCC, LLVM	small, UB-free C functions	66 bugs, almost half miscompilations
Once4All ASPLOS 2026	Z3, cvc5	grammars and term generators	43 confirmed bugs, 40 fixed
G^2Fuzz USENIX Sec 2025	TIFF, MP4, PDF	Python input generators	10 bugs, 3 CVEs

Ni and Li, OOPSLA 2025; Sun, Yang, Zhou, ASPLOS 2026; Zhang et al., USENIX Security 2025; Chen, Dolan-Gavitt, Lin, USENIX Security 2025

The LLM Writes the Generator (4/5)

Call the LLM **once**, offline; then fuzz at full speed with what it wrote

Tool	Target	What the LLM writes	Result
LegoFuzz OOPSLA 2025	GCC, LLVM	small, UB-free C functions	66 bugs, almost half miscompilations
Once4All ASPLOS 2026	Z3, cvc5	grammars and term generators	43 confirmed bugs, 40 fixed
G^2Fuzz USENIX Sec 2025	TIFF, MP4, PDF	Python input generators	10 bugs, 3 CVEs
ELFuzz USENIX Sec 2025	cvc5 and others	the fuzzer itself, evolved	five 0-days in cvc5

Ni and Li, OOPSLA 2025; Sun, Yang, Zhou, ASPLOS 2026; Zhang et al., USENIX Security 2025; Chen, Dolan-Gavitt, Lin, USENIX Security 2025

The LLM Writes the Generator (5/5)

Call the LLM **once**, offline; then fuzz at full speed with what it wrote

Tool	Target	What the LLM writes	Result
LegoFuzz OOPSLA 2025	GCC, LLVM	small, UB-free C functions	66 bugs, almost half miscompilations
Once4All ASPLOS 2026	Z3, cvc5	grammars and term generators	43 confirmed bugs, 40 fixed
G^2Fuzz USENIX Sec 2025	TIFF, MP4, PDF	Python input generators	10 bugs, 3 CVEs
ELFuzz USENIX Sec 2025	cvc5 and others	the fuzzer itself, evolved	five 0-days in cvc5

The LLM supplies **knowledge of the format**; the classic fuzzer supplies **speed**

Ni and Li, OOPSLA 2025; Sun, Yang, Zhou, ASPLOS 2026; Zhang et al., USENIX Security 2025; Chen, Dolan-Gavitt, Lin, USENIX Security 2025

LegoFuzz: An Example (1/4)

Offline: the LLM writes small pieces

```
int b(int y) { return y * 2; }
int a(int x) {
  if (x > 10) return 1;
  else return 0;
}
```

Ni and Li, OOPSLA 2025

LegoFuzz: An Example (2/4)

Offline: the LLM writes small pieces

```
int b(int y) { return y * 2; }
int a(int x) {
  if (x > 10) return 1;
  else return 0;
}
```

Online: no LLM call; add a dependency

```
int a(int x) {
  if (x > b(5)) return 1;
  else return 0;
}
```

Ni and Li, OOPSLA 2025

LegoFuzz: An Example (3/4)

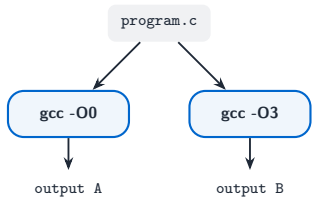
Offline: the LLM writes small pieces

```
int b(int y) { return y * 2; }
int a(int x) {
  if (x > 10) return 1;
  else return 0;
}
```

Online: no LLM call; add a dependency

```
int a(int x) {
  if (x > b(5)) return 1;
  else return 0;
}
```

Oracle: differential testing



A $\neq$ B: miscompilation

Ni and Li, OOPSLA 2025

LegoFuzz: An Example (4/4)

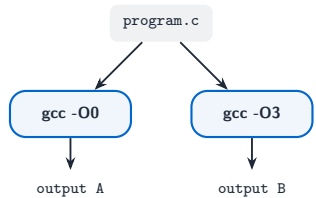
Offline: the LLM writes small pieces

```
int b(int y) { return y * 2; }
int a(int x) {
  if (x > 10) return 1;
  else return 0;
}
```

Online: no LLM call; add a dependency

```
int a(int x) {
  if (x > b(5)) return 1;
  else return 0;
}
```

Oracle: differential testing



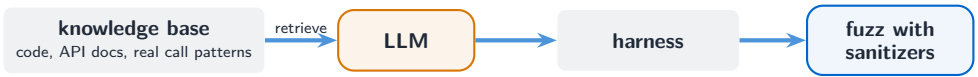
$A \neq B$: miscompilation

Without the call, combining pieces is "almost equivalent" to testing each alone

Ni and Li, OOPSLA 2025

LLM-Written Harnesses (1/3)

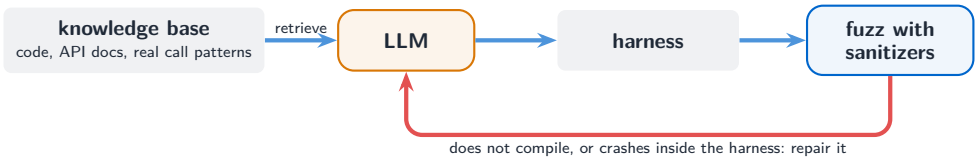
A harness must call the library **correctly**: right order, right arguments, right setup



Liu et al., CCS 2025

LLM-Written Harnesses (2/3)

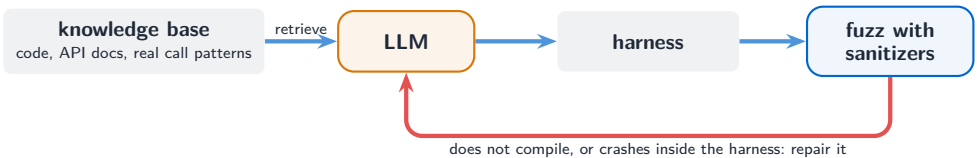
A harness must call the library **correctly**: right order, right arguments, right setup



Liu et al., CCS 2025

LLM-Written Harnesses (3/3)

A harness must call the library **correctly**: right order, right arguments, right setup



PromeFuzz (CCS 2025), 22 projects. Branch coverage compared with:
PromptFuzz **1.50**× OSS-Fuzz-Gen **1.91**× hand-written OSS-Fuzz harnesses **1.40**×
25 new vulnerabilities, 21 confirmed, 3 CVEs

Liu et al., CCS 2025

Syzkaller: Describing System Calls (1/6)

Syzkaller fuzzes the kernel through **system calls**, generated from descriptions in `syzlang`

```
open(file filename, flags flags[open_flags], mode flags[open_mode]) fd
read(fd fd, buf buffer[out], count len[buf])
close(fd fd)
```

one line per system call: its arguments, their types, and its result

github.com/google/syzkaller

Syzkaller: Describing System Calls (2/6)

Syzkaller fuzzes the kernel through **system calls**, generated from descriptions in `syzlang`

```
open(file filename, flags flags[open_flags], mode flags[open_mode]) fd
read(fd fd, buf buffer[out], count len[buf])
close(fd fd)
```

`flags[open_flags]`: only combinations of the real `open` flags

github.com/google/syzkaller

Syzkaller: Describing System Calls (3/6)

Syzkaller fuzzes the kernel through **system calls**, generated from descriptions in `syzlang`

```
open(file filename, flags flags[open_flags], mode flags[open_mode]) fd
read(fd fd, buf buffer[out], count len[buf])
close(fd fd)
```

`buffer[out]`: memory the kernel writes; `len[buf]`: the true length of that buffer

github.com/google/syzkaller

Syzkaller: Describing System Calls (4/6)

Syzkaller fuzzes the kernel through **system calls**, generated from descriptions in `syzlang`

```
open(file filename, flags flags[open_flags], mode flags[open_mode]) fd
read(fd fd, buf buffer[out], count len[buf])
close(fd fd)
```

`fd` is a *resource*: made by `open`, needed by `read` and `close`

github.com/google/syzkaller

Syzkaller: Describing System Calls (5/6)

Syzkaller fuzzes the kernel through **system calls**, generated from descriptions in `syzlang`

```
open(file filename, flags flags[open_flags], mode flags[open_mode]) fd
read(fd fd, buf buffer[out], count len[buf])
close(fd fd)
```

`fd` is a *resource*: made by `open`, needed by `read` and `close`

A generated *program*

```
1 r0 = open(&(0x7f0000000000)="./file0", 0x3, 0x9)
2 read(r0, &(0x7f0000000000), 42)
3 close(r0)
```

- a sequence of system calls with concrete values
- `r0`: the file descriptor that `open` returned

github.com/google/syzkaller

Syzkaller: Describing System Calls (6/6)

Syzkaller fuzzes the kernel through **system calls**, generated from descriptions in `syzlang`

```
open(file filename, flags flags[open_flags], mode flags[open_mode]) fd
read(fd fd, buf buffer[out], count len[buf])
close(fd fd)
```

`fd` is a *resource*: made by `open`, needed by `read` and `close`

A generated *program*

```
1 r0 = open(&(0x7f0000000000)="./file0", 0x3, 0x9)
2 read(r0, &(0x7f0000000000), 42)
3 close(r0)
```

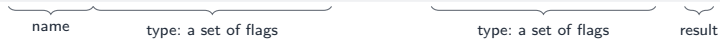
- a sequence of system calls with concrete values
- `r0`: the file descriptor that `open` returned
- valid arguments, in a valid order: the calls get *past* the kernel's checks

github.com/google/syzkaller

syzlang: Reading One Description (1/3)

Each argument is written `name type`; the type says which values are allowed

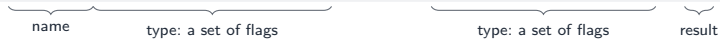
```
open(file filename, flags flags[open_flags], mode flags[open_mode]) fd
```



syzlang: Reading One Description (2/3)

Each argument is written `name type`; the type says which values are allowed

```
open(file filename, flags flags[open_flags], mode flags[open_mode]) fd
```




```
open_flags = O_WRONLY, O_RDWR, O_APPEND, FASYNC, O_CLOEXEC, O_CREAT, O_DIRECT, O_EXCL, ..., O_TRUNC
open_mode  = S_IRUSR, S_IWUSR, S_IXUSR, S_IRGRP, S_IWGRP, S_IXGRP, S_IROTH, S_IWOTH, S_IXOTH
```

syzlang: Reading One Description (3/3)

Each argument is written `name type`; the type says which values are allowed

```
open(file filename, flags flags[open_flags], mode flags[open_mode]) fd
```

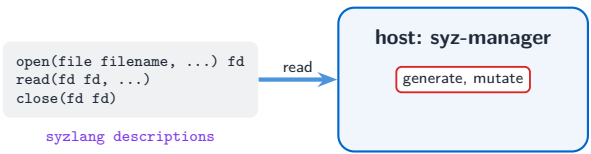
name — type: a set of flags — type: a set of flags — result

```
open_flags = O_WRONLY, O_RDWR, O_APPEND, FASYNC, O_CLOEXEC, O_CREAT, O_DIRECT, O_EXCL, ..., O_TRUNC
open_mode  = S_IRUSR, S_IWUSR, S_IXUSR, S_IRGRP, S_IWGRP, S_IXGRP, S_IROTH, S_IWOTH, S_IXOTH
```

```
r0 = open(&(0x7f0000000000)="./file0", 0x3, 0x9)
```

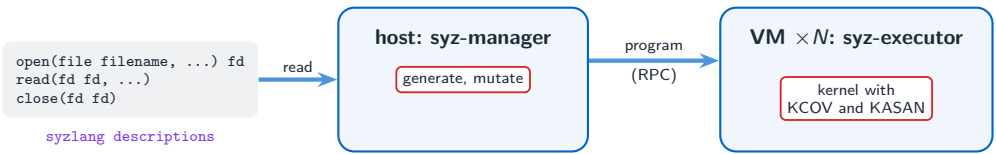
0x3 = O_WRONLY (1) | O_RDWR (2)
0x9 = S_IXGRP (8) | S_IXOTH (1)

Syzkaller: The Fuzzing Loop (1/5)



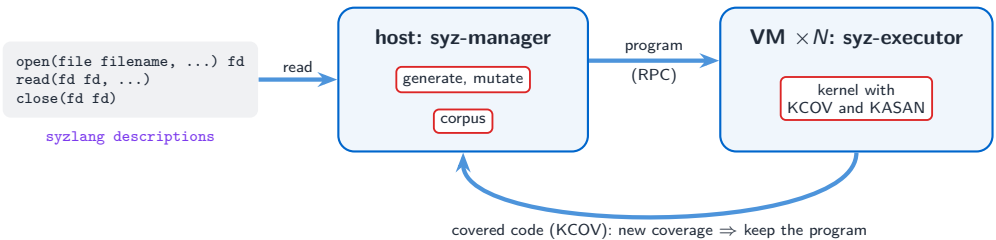
github.com/google/syzkaller

Syzkaller: The Fuzzing Loop (2/5)



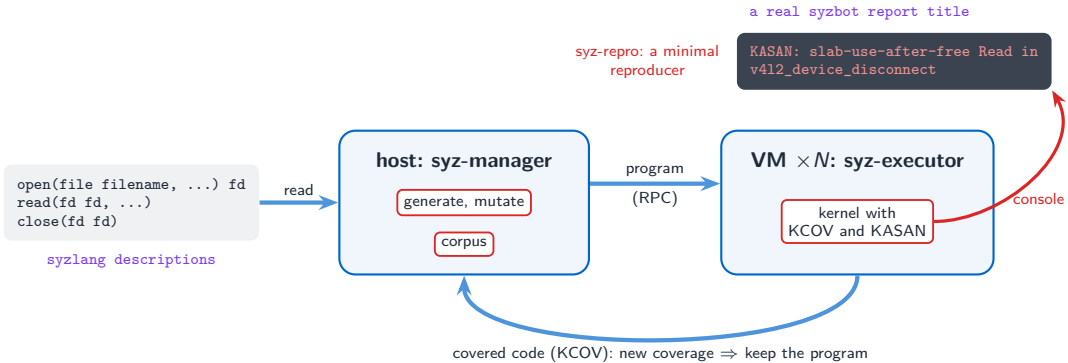
github.com/google/syzkaller

Syzkaller: The Fuzzing Loop (3/5)



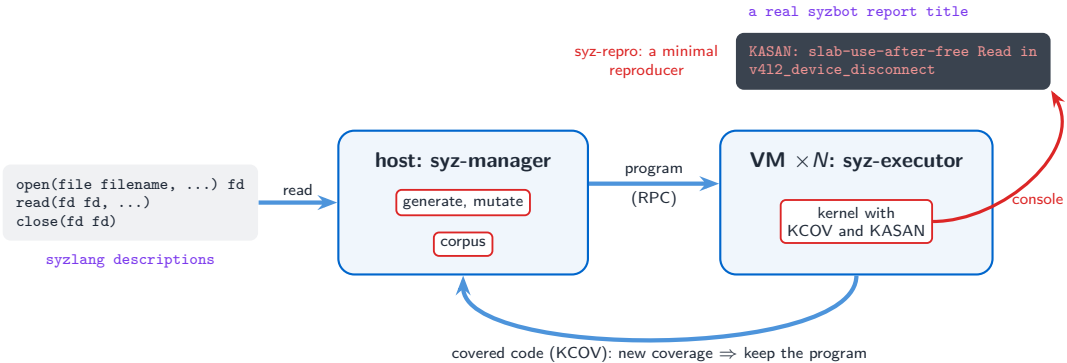
github.com/google/syzkaller

Syzkaller: The Fuzzing Loop (4/5)



github.com/google/syzkaller

Syzkaller: The Fuzzing Loop (5/5)



syzbot runs this around the clock on Linux: 7,615 fixed upstream bugs

github.com/google/syzkaller

Kernels and Protocols (1/4)

Kernel fuzzing (Syzkaller)

- **KernelGPT** (ASPLOS 2025): the LLM reads driver code and writes the descriptions: 532 new syscall descriptions; 24 new bugs, 11 CVEs

Yang, Zhao, Zhang, ASPLOS 2025; Zhang, Li, Liang, Chen, ISSTA 2025; Meng, Mirchev, Böhme, Roychoudhury, NDSS 2024

Kernels and Protocols (2/4)

Kernel fuzzing (Syzkaller)

- **KernelGPT** (ASPLOS 2025): the LLM reads driver code and writes the descriptions: 532 new syscall descriptions; 24 new bugs, 11 CVEs
- **SyzGPT** (ISSTA 2025): seeds for *rarely used* syscalls; 87.84% valid, 26 new bugs

Yang, Zhao, Zhang, ASPLOS 2025; Zhang, Li, Liang, Chen, ISSTA 2025; Meng, Mirchev, Böhme, Roychoudhury, NDSS 2024

Kernels and Protocols (3/4)

Kernel fuzzing (Syzkaller)

- **KernelGPT** (ASPLOS 2025): the LLM reads driver code and writes the descriptions: 532 new syscall descriptions; 24 new bugs, 11 CVEs
- **SyzGPT** (ISSTA 2025): seeds for *rarely used* syscalls; 87.84% valid, 26 new bugs

Protocol fuzzing

A server accepts messages only in the right *state*

- **ChatAFL** (NDSS 2024): the LLM knows the protocol from its RFC; writes message grammars and seeds, and asks for messages that reach new states
- 47.6% more state transitions than AFLNet; 9 new vulnerabilities

Yang, Zhao, Zhang, ASPLOS 2025; Zhang, Li, Liang, Chen, ISSTA 2025; Meng, Mirchev, Böhme, Roychoudhury, NDSS 2024

Kernels and Protocols (4/4)

Kernel fuzzing (Syzkaller)

- **KernelGPT** (ASPLOS 2025): the LLM reads driver code and writes the descriptions: 532 new syscall descriptions; 24 new bugs, 11 CVEs
- **SyzGPT** (ISSTA 2025): seeds for *rarely used* syscalls; 87.84% valid, 26 new bugs

Protocol fuzzing

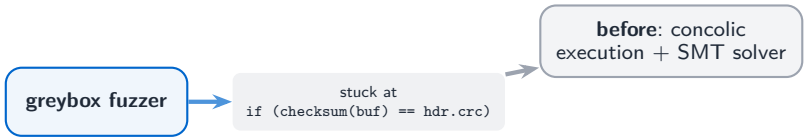
A server accepts messages only in the right *state*

- **ChatAFL** (NDSS 2024): the LLM knows the protocol from its RFC; writes message grammars and seeds, and asks for messages that reach new states
- 47.6% more state transitions than AFLNet; 9 new vulnerabilities

Again: the LLM knows the **interface**, from code and documents; the fuzzer **explores**

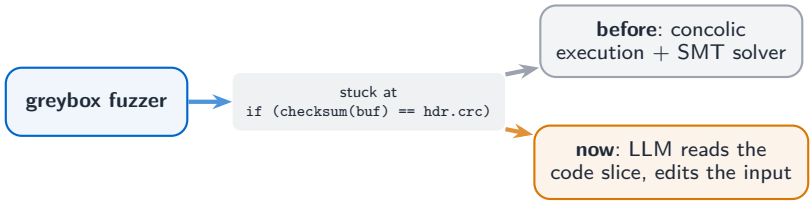
Yang, Zhao, Zhang, ASPLOS 2025; Zhang, Li, Liang, Chen, ISSTA 2025; Meng, Mirchev, Böhme, Roychoudhury, NDSS 2024

The LLM Instead of the Solver (1/3)



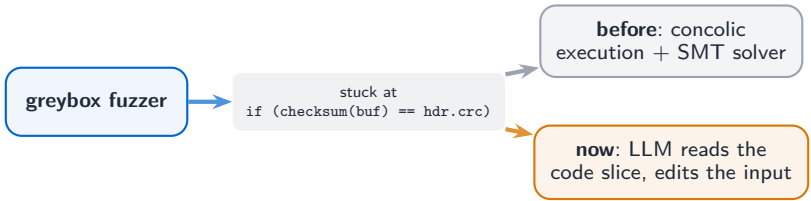
Meng, Duck, Roychoudhury, IEEE TSE 2026; Tu et al., IEEE S&P 2026

The LLM Instead of the Solver (2/3)



Meng, Duck, Roychoudhury, IEEE TSE 2026; Tu et al., IEEE S&P 2026

The LLM Instead of the Solver (3/3)



HyLLfuzz TSE 2026	LLM replaces the solver	+31.43% branch coverage over CoFuzz; 3–19× faster than concolic
Cottontail S&P 2026	LLM solves *and* keeps the input parseable	+41.32% branch coverage; 6 new CVEs

Meng, Duck, Roychoudhury, IEEE TSE 2026; Tu et al., IEEE S&P 2026

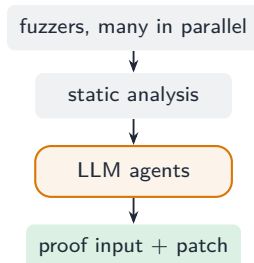
At Scale: DARPA's AI Cyber Challenge (1/4)

- 7 finalist systems: find *and patch* vulnerabilities, without humans
- About 143 hours, 53 challenge projects from 24 OSS-Fuzz repositories

Zhang et al., USENIX Security 2026

At Scale: DARPA's AI Cyber Challenge (2/4)

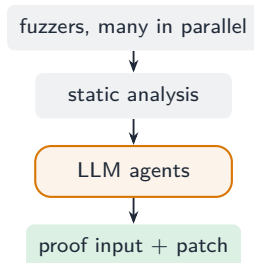
- 7 finalist systems: find *and patch* vulnerabilities, without humans
- About 143 hours, 53 challenge projects from 24 OSS-Fuzz repositories
- Fuzzing + static analysis + LLM agents



Zhang et al., USENIX Security 2026

At Scale: DARPA's AI Cyber Challenge (3/4)

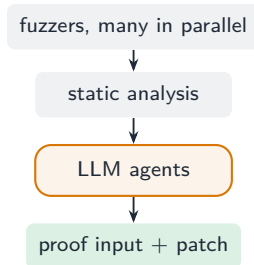
- 7 finalist systems: find *and patch* vulnerabilities, without humans
- About 143 hours, 53 challenge projects from 24 OSS-Fuzz repositories
- Fuzzing + static analysis + LLM agents
- **25 distinct 0-days** in 10 open-source projects; 12 patched



Zhang et al., USENIX Security 2026

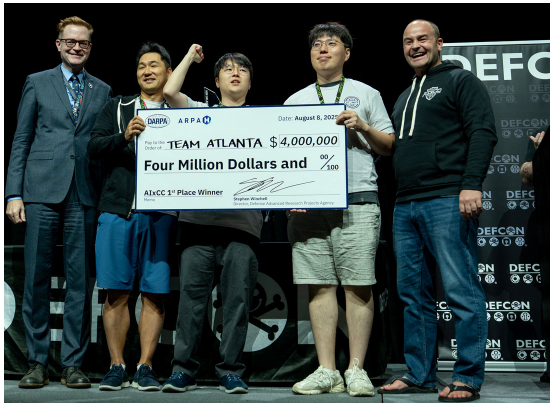
At Scale: DARPA's AI Cyber Challenge (4/4)

- 7 finalist systems: find *and patch* vulnerabilities, without humans
- About 143 hours, 53 challenge projects from 24 OSS-Fuzz repositories
- Fuzzing + static analysis + LLM agents
- **25 distinct 0-days** in 10 open-source projects; 12 patched
- LLM spending closely tracked the final score



Zhang et al., USENIX Security 2026

AIxCC Final: The Winners (1/3)

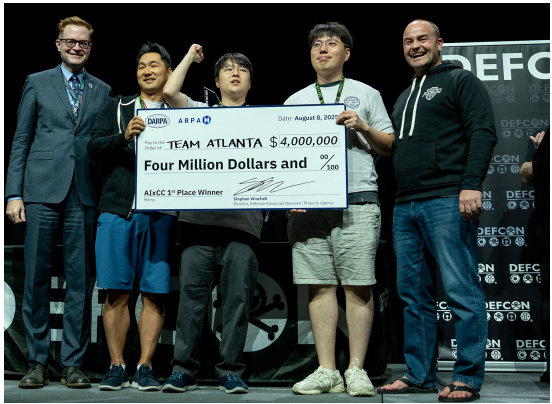




1st: Team Atlanta, $4,000,000, DEF CON 33, 8 August 2025

DARPA, AIxCC results, 2025; Kim et al., ATLANTIS technical report, 2025; photos: DARPA | Matt Dombro

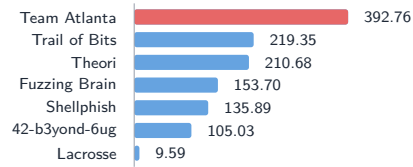
AIxCC Final: The Winners (2/3)



1st: Team Atlanta, $4,000,000, DEF CON 33, 8 August 2025

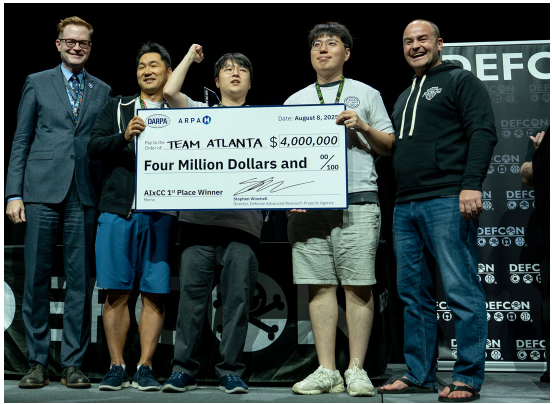


Final scores



DARPA, AIxCC results, 2025; Kim et al., ATLANTIS technical report, 2025; photos: DARPA | Matt Dombro

AIxCC Final: The Winners (3/3)



1st: Team Atlanta, $4,000,000, DEF CON 33, 8 August 2025

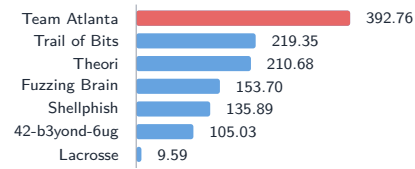


2nd: Trail of Bits, $3,000,000



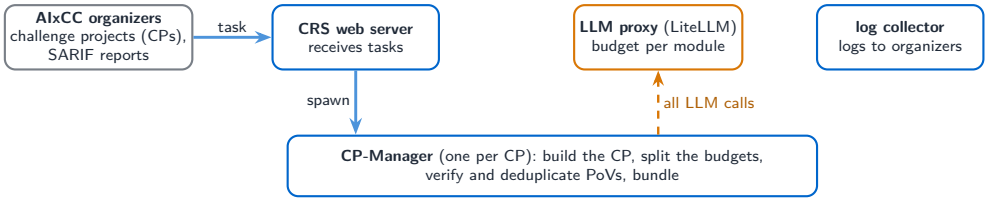
3rd: Theori, $1,500,000

Final scores



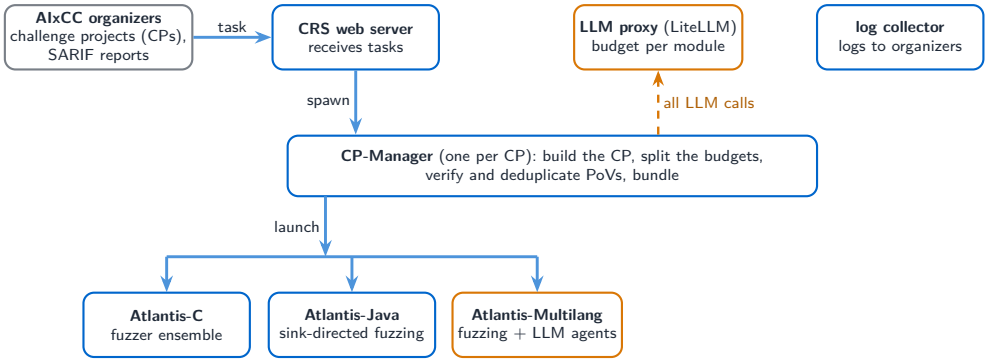
DARPA, AIxCC results, 2025; Kim et al., ATLANTIS technical report, 2025; photos: DARPA | Matt Dombro

Atlantis: Overall Architecture (1/6)



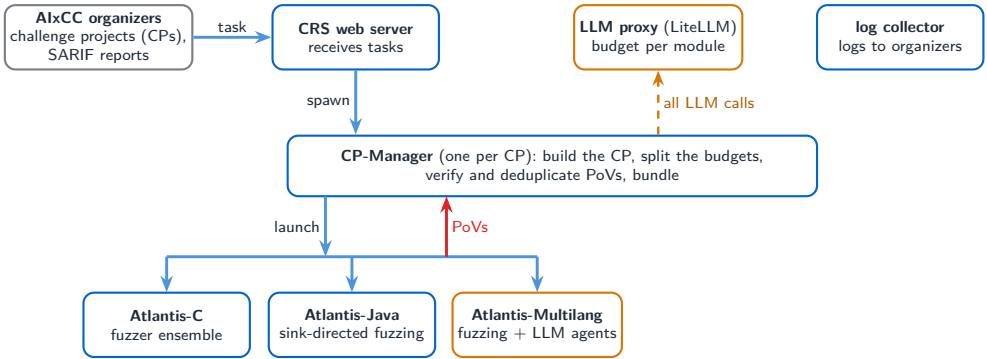
Kim et al., ATLANTIS technical report, 2025 (Figure 1, redrawn)

Atlantis: Overall Architecture (2/6)



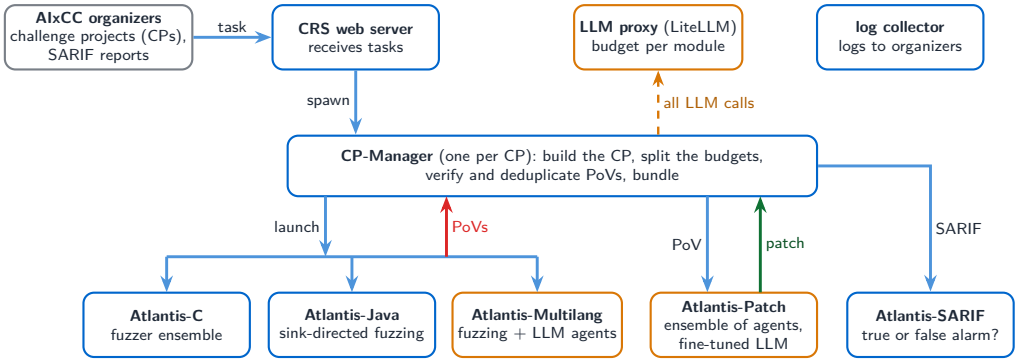
Kim et al., ATLANTIS technical report, 2025 (Figure 1, redrawn)

Atlantis: Overall Architecture (3/6)



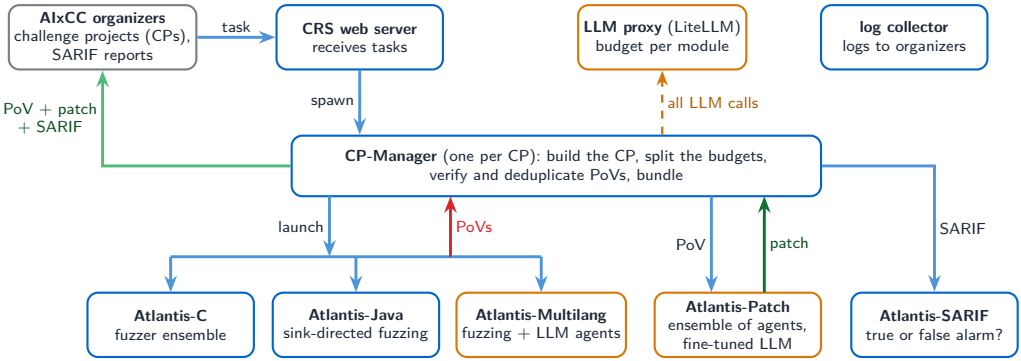
Kim et al., ATLANTIS technical report, 2025 (Figure 1, redrawn)

Atlantis: Overall Architecture (4/6)



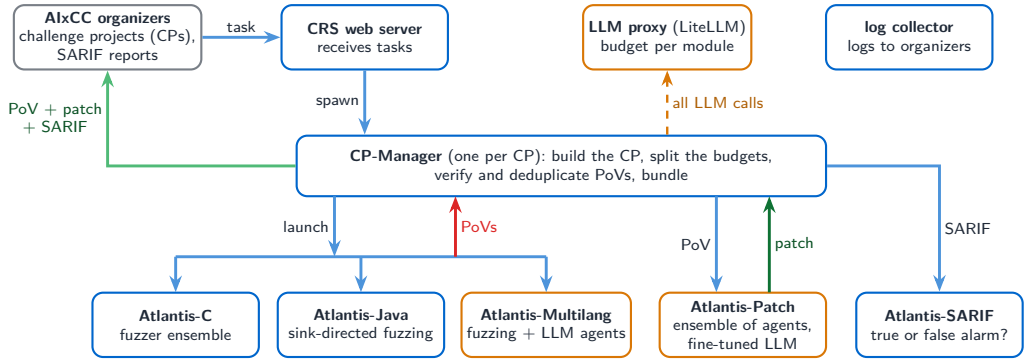
Kim et al., ATLANTIS technical report, 2025 (Figure 1, redrawn)

Atlantis: Overall Architecture (5/6)



Kim et al., ATLANTIS technical report, 2025 (Figure 1, redrawn)

Atlantis: Overall Architecture (6/6)



Kubernetes on Azure, set up with Terraform. LLM budget: 37.5% C or Java, 37.5% Multilang, 25% Patch, $500 SARIF

Kim et al., ATLANTIS technical report, 2025 (Figure 1, redrawn)

Taesoo Kim: From AIxCC to Microsoft (1/2)



photo: Microsoft

Taesoo Kim

Vice President, Agentic Security, Microsoft
Professor, Georgia Tech
led Team Atlanta, winner of AIxCC

Microsoft's MDASH is built on the lessons of **Team Atlanta**.

T. Kim, Microsoft Security Blog, May 2026

Taesoo Kim: From AIxCC to Microsoft (2/2)



photo: Microsoft

Taesoo Kim

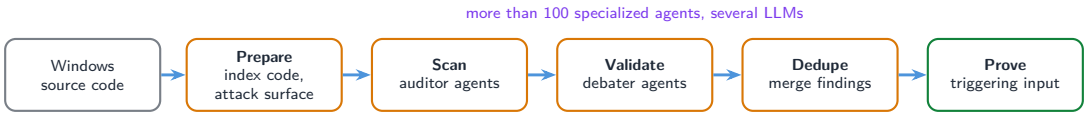
Vice President, Agentic Security, Microsoft
Professor, Georgia Tech
led Team Atlanta, winner of AIxCC

Microsoft's MDASH is built on the lessons of **Team Atlanta**.

"The model is one input.
The system is the product."

T. Kim, Microsoft Security Blog, May 2026

MDASH: Finding Bugs in Windows



16 new Windows vulnerabilities

critical RCE	component	
CVE-2026-33827	`tcpip.sys`	IPv4, use-after-free
CVE-2026-33824	`ikeext.dll`	IKE (VPN), double free
CVE-2026-41089	`netlogon.dll`	Netlogon
CVE-2026-41096	`dnsapi.dll`	DNS client

- planted-bug test driver: **21 of 21** found, 0 false positives
- CyberGym benchmark: **88.45%**, top of the leaderboard

T. Kim, Microsoft Security Blog, May 2026; CSO Online, May 2026